

Finding $f(n)$ and Proving Θ Bounds

Objective

In this handout, we analyze two simple algorithms step by step: For each, we first find $f(n)$ (the exact count of operations), then prove the asymptotic Θ bound.

Question 1: Nested Loops

Code:

```
for i = 1 to n
  for j = 1 to i
    print(i * j)
```

Step 1: Counting Basic Operations

The inner statement executes once for every (i, j) pair such that $1 \leq j \leq i$ and $1 \leq i \leq n$:

$$f(n) = \sum_{i=1}^n i = 1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}.$$

Step 2: Simplify

$$f(n) = \frac{1}{2}n^2 + \frac{1}{2}n.$$

Step 3: Prove Θ Bound

We want to show that $f(n) \in \Theta(n^2)$.

By definition, $f(n) \in \Theta(g(n))$ if there exist constants $c_1, c_2 > 0$ and n_0 such that:

$$c_1 g(n) \leq f(n) \leq c_2 g(n), \quad \forall n \geq n_0.$$

Take $g(n) = n^2$. For all $n \geq 1$:

$$n^2 \leq f(n) \leq 2n^2.$$

Thus $c_1 = 1$, $c_2 = 2$, and $n_0 = 1$ work.

Result

$$f(n) \in \Theta(n^2)$$

Question 2: Logarithmic Loop

Code:

```
i = 1
while i <= n
    print(i)
    i = i * 2
```

Step 1: Counting Operations

Values of i take the form:

$$1, 2, 4, 8, \dots, 2^k \leq n.$$

Thus, the number of iterations is approximately:

$$k = \lfloor \log_2 n \rfloor + 1.$$

So:

$$f(n) = \lfloor \log_2 n \rfloor + 1.$$

Step 2: Simplify

Ignoring the floor and constant:

$$f(n) = \log_2 n + 1.$$

Step 3: Prove Θ Bound

For $n \geq 2$:

$$\log_2 n \leq f(n) \leq \log_2 n + 1 \leq 2 \log_2 n.$$

Thus, with $c_1 = 1$, $c_2 = 2$, and $n_0 = 2$:

$$c_1 \log n \leq f(n) \leq c_2 \log n.$$

Result

$$f(n) \in \Theta(\log n)$$